OS ORANGE Report

Name: Rottela Haritej
SRN: PES1UG24AM231
Repository: https://github.com/RHaritej/PES1UG24AM231-pes-vcs

Setup Commands

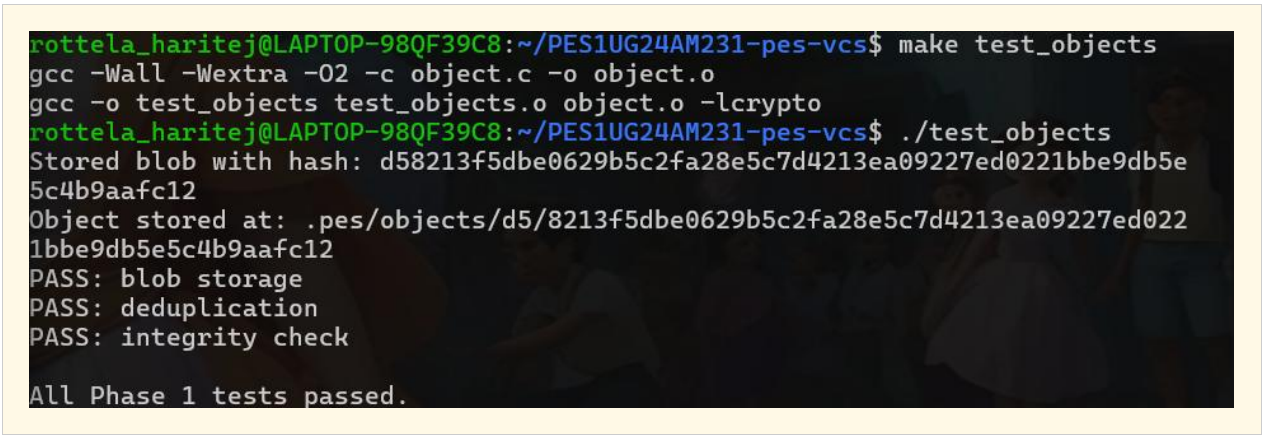
```
sudo apt update && sudo apt install -y gcc build-essential libssl-dev
export PES_AUTHOR="Your Name <PES1UG24AM231>"
git clone <your-repo-url>
cd PES1UG24AM231-pes-vcs
```

Phase 1 — Object Storage

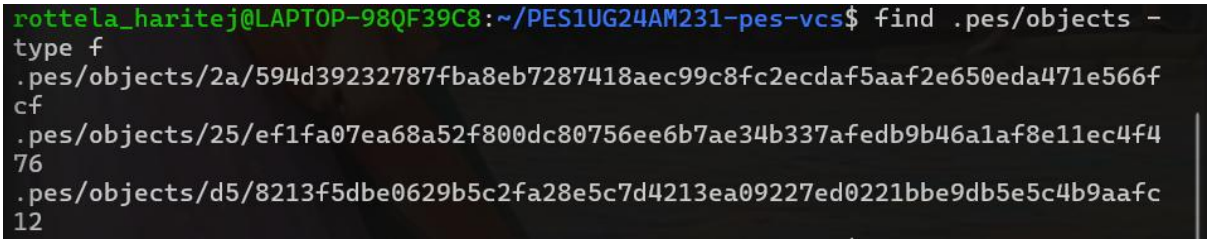
Commands Run

```
make test_objects
./test_objects
./pes init
find .pes/objects -type f
```

Screenshot 1A — ./test_objects output



Screenshot 1B — Sharded object directory

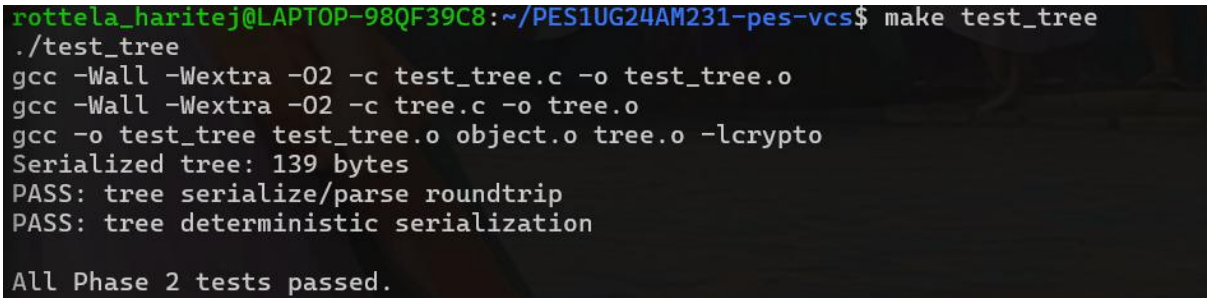


Phase 2 — Tree Objects

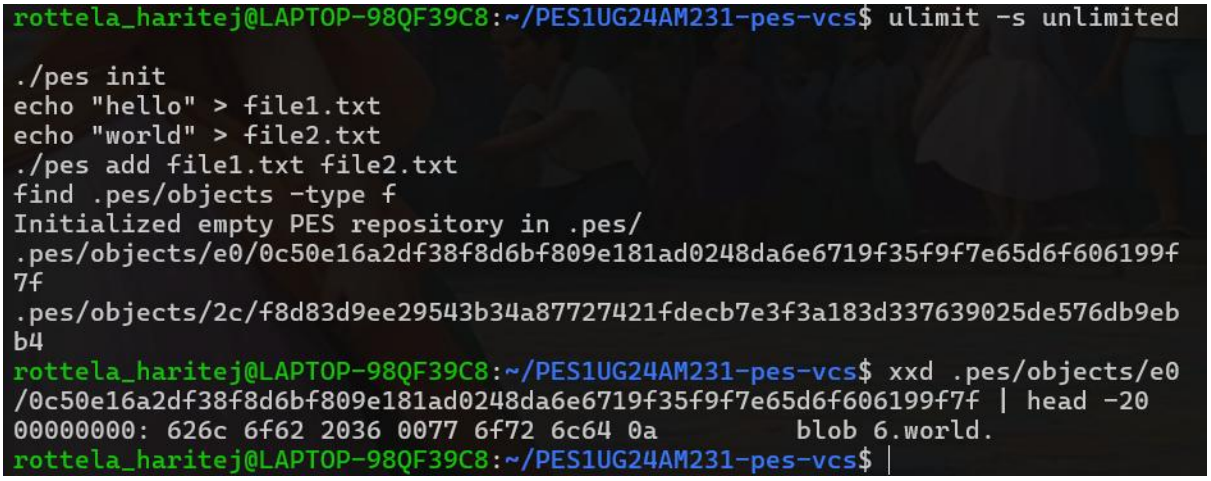
Commands Run

```
make test_tree
./test_tree
ulimit -s unlimited
./pes init
./pes add file1.txt file2.txt
find .pes/objects -type f
xxd .pes/objects/XX/YYYY... | head -20
```

Screenshot 2A — ./test_tree output



Screenshot 2B — Raw binary tree object (xxd)

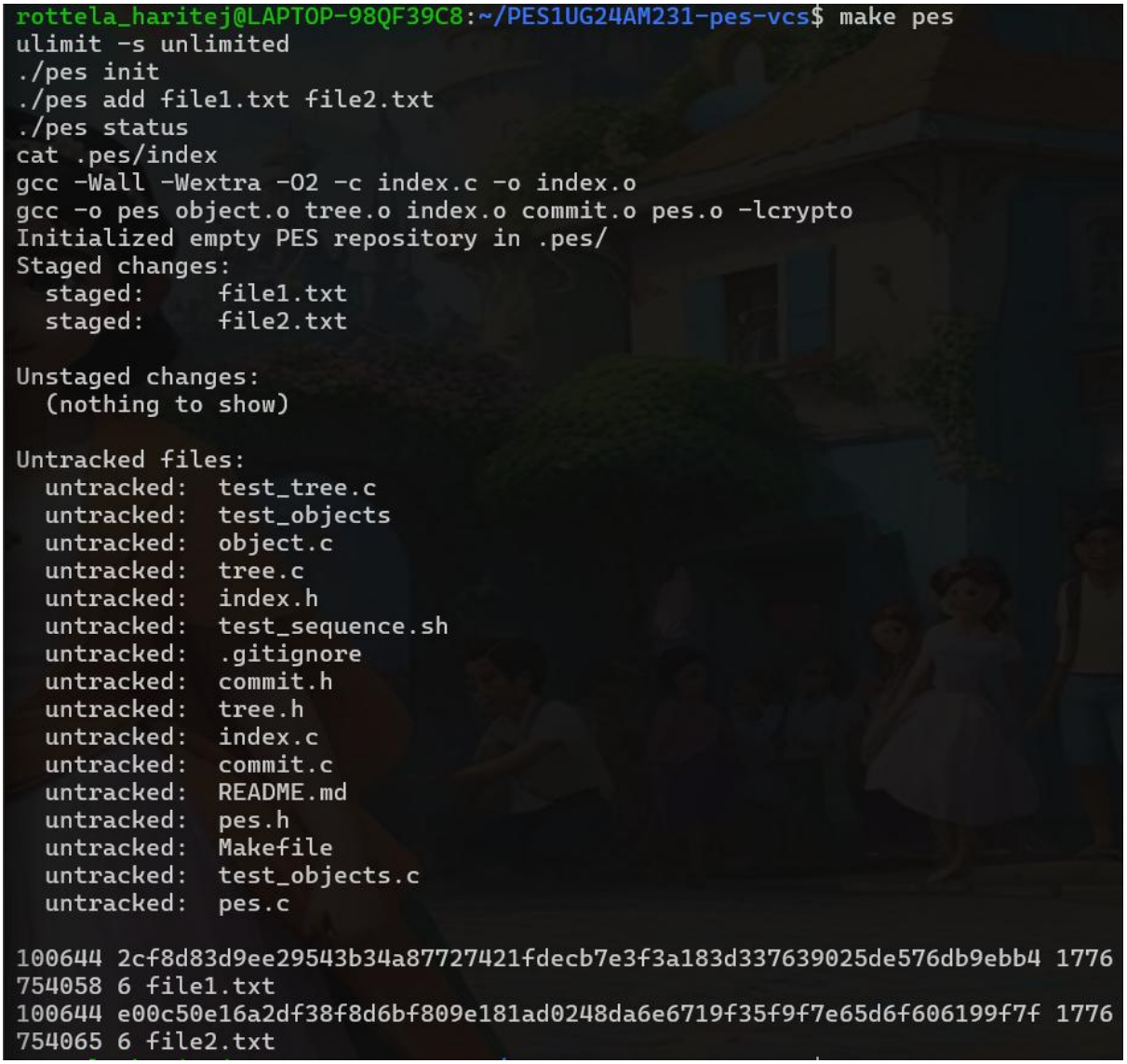


Phase 3 — Index (Staging Area)

Commands Run

```
ulimit -s unlimited
./pes init
echo "hello" > file1.txt
echo "world" > file2.txt
./pes add file1.txt file2.txt
./pes status
cat .pes/index
```

Screenshot 3A — pes init -> pes add -> pes status



Screenshot 3B — cat .pes/index

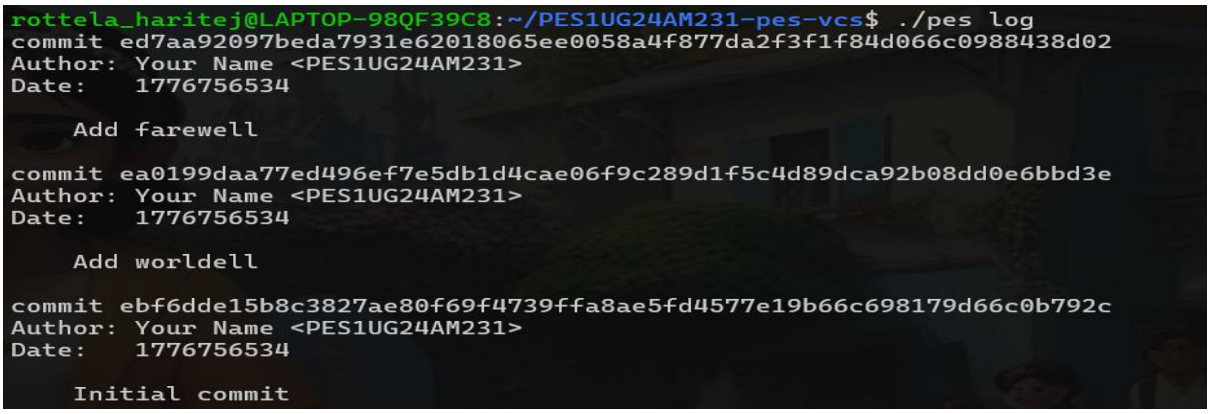
Phase 4 — Commits and History

Commands Run

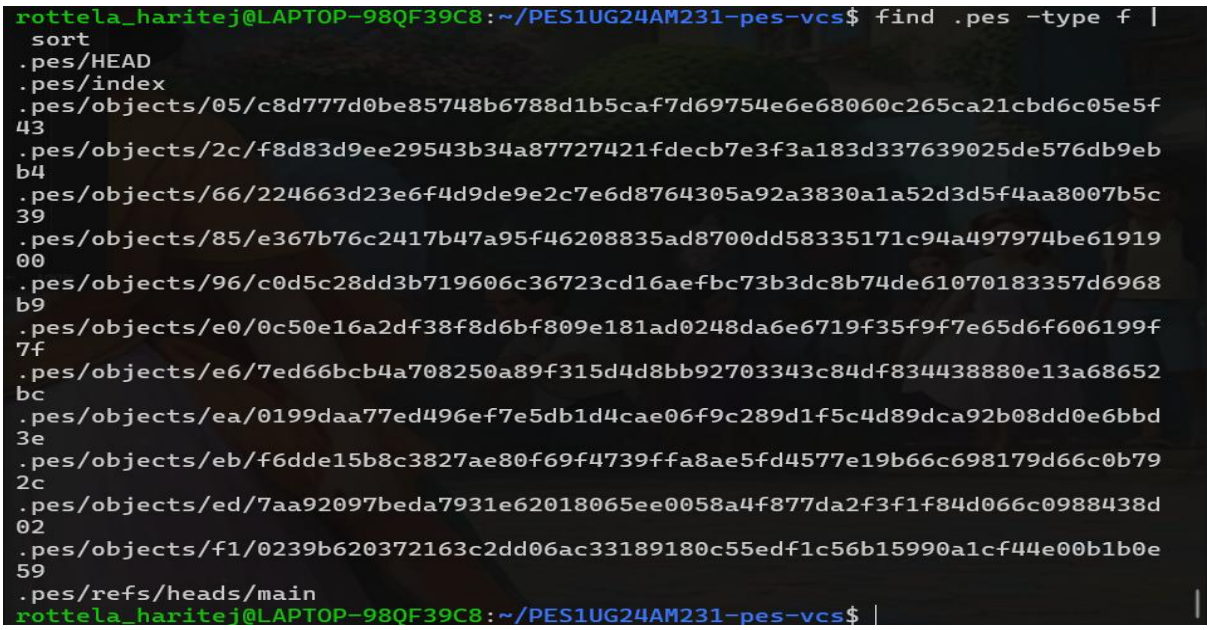
```
ulimit -s unlimited
./pes init
echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"
echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"
echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
./pes log
```

```
find .pes -type f | sort
cat .pes/refs/heads/main
cat .pes/HEAD
```

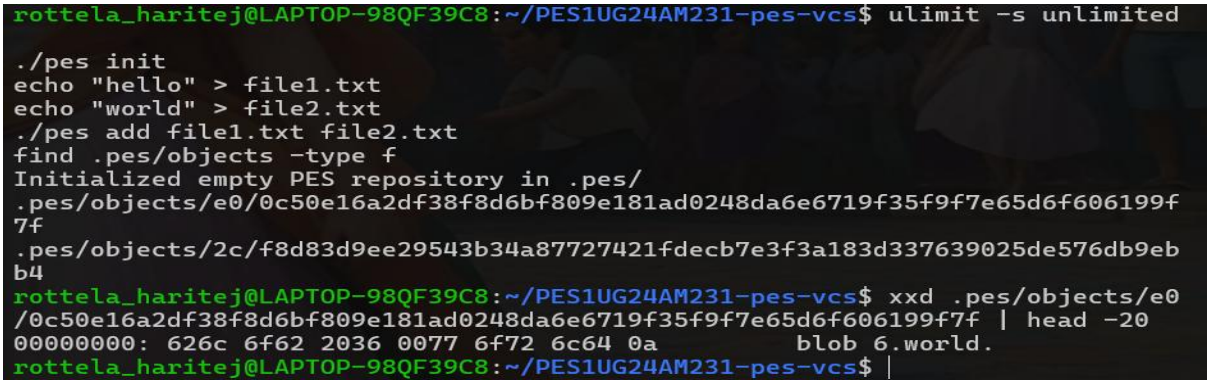
Screenshot 4A — pes log with three commits



Screenshot 4B — find .pes -type f | sort



Screenshot 4C — Reference chain



Phase 5 — Branching Analysis

Q5.1 — How would you implement pes checkout <branch>?

To implement pes checkout <branch>, three things must happen:

Files that change in .pes/:
HEAD is rewritten to ref: refs/heads/<branch>. No other .pes/ files change — the objects and refs are untouched.

Working directory update:
1. Read the current HEAD commit and walk its tree recursively to get the full list of current files and their blob hashes.
2. Read the target branch's commit and walk its tree recursively to get the full list of target files.
3. For each file in the target tree: read the blob from the object store and write it to the working directory.
4. For each file in the current tree that does NOT exist in the target tree: delete it from the working directory.

What makes it complex:
Trees are nested — you must recursively walk subtrees and reconstruct full paths (e.g., the src/ subtree contains main.c, so the working directory file is src/main.c). Files that are identical in both trees (same blob hash) can be skipped for efficiency. Subdirectories that exist in the current branch but not the target must be removed, but only if they are empty after removing tracked files. If the user has uncommitted changes to a tracked file that differs between branches, checkout must refuse.

Q5.2 — How to detect a dirty working directory conflict?

To detect whether a file would cause a conflict during checkout, using only the index and object store:

1. For each file in the index, check if the working directory version differs from the staged version by comparing mtime and size (fast path) or by re-hashing the file (slow path). If they differ, the file has local changes that were never staged.
2. Compare the blob hash in the index for that file against the blob hash in the HEAD tree for that same file. If they differ, the file is staged but not committed.
3. Look up the same file path in the target branch's tree. If the blob hash there differs from what is currently on disk, and the file has local modifications (steps 1 or 2), then there is a conflict.
4. If any such conflict exists, print an error and abort. Only proceed if all modified files are identical between the two branches, or if the file does not exist in the target branch and has no local changes.

Q5.3 — What happens in detached HEAD, and how to recover?

In detached HEAD state, .pes/HEAD contains a raw commit hash directly (e.g., a1b2c3...) instead of ref: refs/heads/main.

What happens when you commit in detached HEAD:
Each new commit is created normally and points to the previous commit as its parent. But head_update() writes the new commit hash directly into HEAD (since it is not a ref). No branch pointer is updated — the new commits exist in the object store but no branch points to them.

How the commits become lost:
If you then run pes checkout main, HEAD is rewritten to ref: refs/heads/main. The commits made in detached state are now unreachable — no branch or HEAD points to them. They will be deleted by garbage collection.

How to recover:
If you remember the hash of the last detached commit (from your terminal history), you can create a new branch pointing to it:

```
echo "a1b2c3d4..." > .pes/refs/heads/recovered
```

If you do not remember the hash and no reflog is implemented, the commits are unrecoverable once GC runs. This is why Git implements a reflog — it records every value HEAD has ever pointed to, making recovery always possible.

Phase 6 — Garbage Collection Analysis

Q6.1 — Algorithm to find and delete unreachable objects

Algorithm (Mark and Sweep):

Mark phase — build the reachable set:
1. Start with every file in .pes/refs/heads/ — each contains a commit hash. Add all to a hash set (reachable set).
2. For each reachable commit: add its tree hash. If it has a parent, add the parent hash and process recursively (walk the entire history chain).
3. For each reachable tree: read it, add every blob and subtree hash it contains. Recurse into subtrees.
4. Continue until no new hashes are added to the set.

Sweep phase — delete unreachable objects:
Walk every file under .pes/objects/XX/YYYY..., reconstruct each object's hash from its path, and delete any file whose hash is NOT in the reachable set.

Data structure:
A hash set for O(1) average lookup. A simple sorted char[65] array with bsearch would work for small repos.

Estimate for 100,000 commits and 50 branches:
Follow 50 branch tips, walk all 100,000 commits, and process each commit's tree (averaging 5-10 blobs and subtrees each). Roughly 500,000 to 1,000,000 total objects to visit during the mark phase.

Q6.2 — Race condition between GC and commit

The race condition:
1. A commit operation calls tree_from_index() — the tree object is written to .pes/objects/ but HEAD has not been updated yet. The new tree is unreachable from any branch.
2. GC starts at this exact moment. It walks from HEAD and finds the new tree unreachable (HEAD still points to the old commit). GC deletes it.
3. The commit operation then calls object_write(OBJ_COMMIT, ...) referencing the now-deleted tree hash. The commit is written and HEAD is updated, but the tree it points to no longer exists. The repository is permanently corrupt.

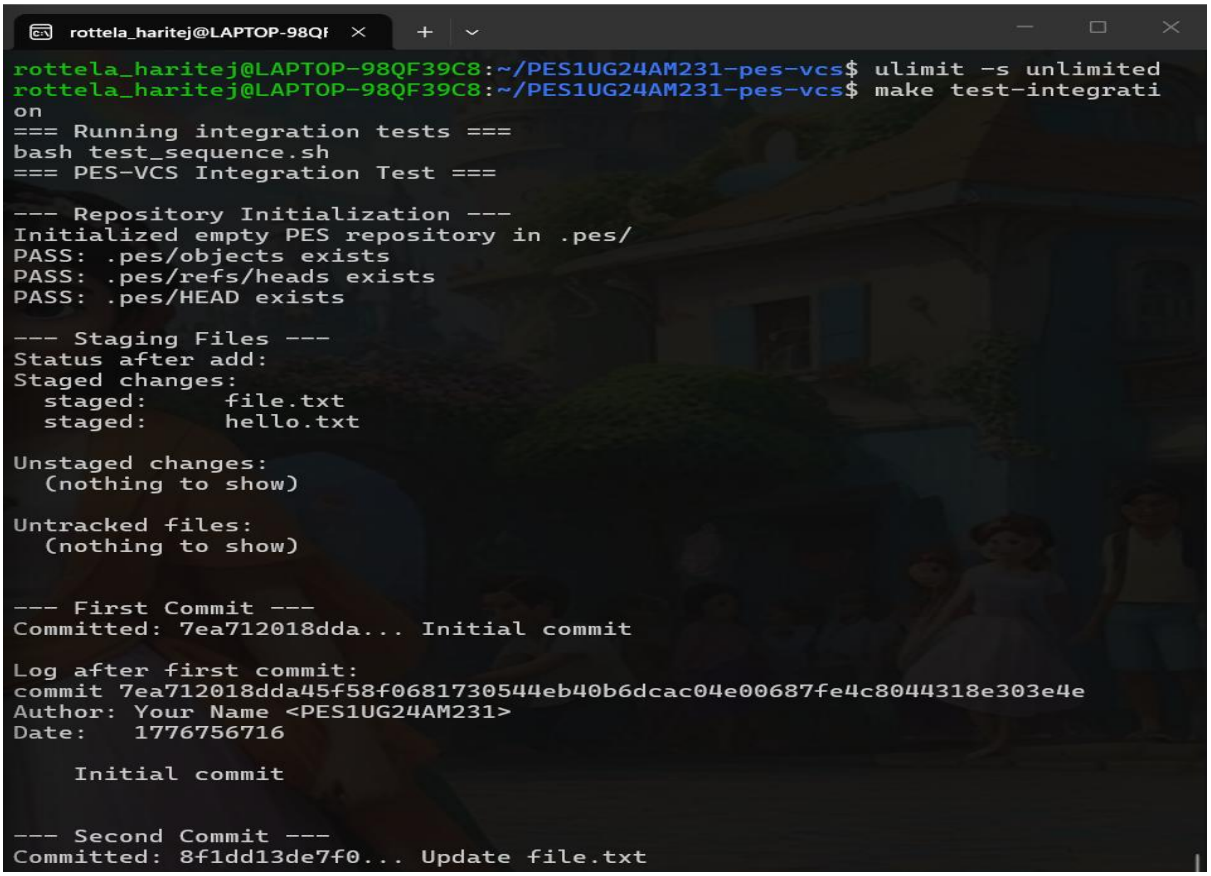
How Git avoids this:
Grace period: Git's GC never deletes objects newer than 2 weeks old. In-progress operations complete in milliseconds, so recently created objects are always safe.
Lock files: Git uses .git/gc.pid and operation-specific lock files so GC and write operations cannot run simultaneously.
Loose object safety: Objects are written atomically (temp file + rename), so a partially written object is never visible to GC.

Integration Test

Commands Run

```
ulimit -s unlimited
make test-integration
```

Screenshot — Full integration test output




rottela_haritej@LAPTOP-98QF
--- Third Commit ---
Committed: 7cce8067f681... Add farewell
--- Full History ---
commit 7cce8067f6815a37d2edc7f1d35145734dd2d55835481c4496088e002a3960a2
Author: Your Name <PES1UG24AM231>
Date: 1776756716
Add farewell
commit 8f1dd13de7f0e548df3c51024b947114d0cc82c6e1283b39d61e13f06ac59735
Author: Your Name <PES1UG24AM231>
Date: 1776756716
Update file.txt
commit 7ea712018dda45f58f0681730544eb40b6dcac04e00687fe4c8044318e303e4e
Author: Your Name <PES1UG24AM231>
Date: 1776756716
Initial commit
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
7cce8067f6815a37d2edc7f1d35145734dd2d55835481c4496088e002a3960a2
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/7c/ce8067f6815a37d2edc7f1d35145734dd2d55835481c4496088e002a3960a2
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/7c/ce8067f6815a37d2edc7f1d35145734dd2d55835481c4496088e002a3960a2
.pes/objects/7e/a712018dda45f58f0681730544eb40b6dcac04e00687fe4c8044318e303e4e
.pes/objects/8f/1dd13de7f0e548df3c51024b947114d0cc82c6e1283b39d61e13f06ac59735
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
=== All integration tests completed ===